

The most common use of first-class functions is so you can pass functions to other functions. For example, we can define a `callTwice()` function, which can be passed a function that needs to be called twice:

```
>>> def callTwice(func):
...     func()
...     func()
...
>>> callTwice(spam)
Spam! Spam! Spam!
Spam! Spam! Spam!
```

You could just write `spam()` twice in your source code. But you can pass the `callTwice()` function to any function at runtime rather than having to type the function call twice into the source code beforehand.

Commonly Confused Terms

Technical jargon is confusing enough, especially for terms that have related but distinct definitions. To make matters worse, languages, operating systems, and fields in computing might use different terms to mean the same thing or the same terms to mean different things. To communicate clearly with other programmers, you'll need to learn the difference between the following terms.

Statements vs. Expressions

Expressions are the instructions made up of operators and values that evaluate to a single value. A value can be a variable (which contains a value) or a function call (which returns a value). So, `2 + 2` is an expression that evaluates down to the single value of 4. But `len(myName) > 4` and `myName.isupper()` or `myName == 'Zophie'` are expressions as well. A value by itself is also an expression that evaluates to itself.

Statements are, effectively, all other instructions in Python. These include `if` statements, `for` statements, `def` statements, `return` statements, and so on. Statements do *not* evaluate to a value. Some statements can include